support.py - Classifier and Escalation Packet

What this file does (big picture)

support.py has two jobs now:
1) Classify a customer question into one of six support lanes
2) Build an EscalationPacket when a human must take over

Important boundary: this file does NOT look up orders. When classification returns Order Help, main.py calls orders.py for status and tracking.

Because keyword lists are long, this guide uses line ranges for clue lists.

Professional habit: check costly mistakes first (legal, refunds, escalation) before vaguer matches like sleep tips.

New words you will see in this file

constant	A fixed name in CAPITALS that we do not change while running.
dictionary	A lookup table of key/value pairs written with { }.
dataclass	A typed form/container for related fields (EscalationPacket).
function	A named reusable block. def starts one; return hands an answer back.
Optional	A type hint meaning a value may be present or missing (None).
uuid	A unique ID generator so each escalation can be tracked.
any()	True if at least one clue matched inside the customer text.

Flow while the program runs

classify_request: clean text, then check escalation, returns, account, order, product, sleep; else escalate.

If escalated: infer reason, map priority, build EscalationPacket for operators.

support.py - Classifier and Escalation Packet

1-11

```
"""Support helpers... Order actions live in orders.py..."""
```

Plain English:
Module docstring. Updated to state the orders.py boundary clearly: classify here, order actions elsewhere.

13-18

```
from __future__ import annotations + imports
```

Plain English:
Bring in dataclass tools, timestamps, Optional, and uuid for unique escalation IDs.

20-27

```
ORDER_HELP = "order_help" (and five more categories)
```

Plain English:
Six lane constants. ORDER_HELP is only a label here; lookup logic is not in this file.

29-36

```
CATEGORY_LABELS = { ... }
```

Plain English:
Map machine codes to human-readable names for printing.

38-50

```
PRIORITY_* and REASON_* constants
```

Plain English:
Operator priority levels and why-we-escalated codes for routing and reporting.

52-57

```
SLEEP_DISCLAIMER = (...)
```

Plain English:
Approved wellness disclaimer text used for sleep-guidance replies.

60-102

```
@dataclass class EscalationPacket: ...
```

Plain English:
The handoff package for humans: conversation id, question, reason, priority, transcript, notes. to_dict() and summary() help logging and display.

105-118

```
def classify_request(question: str) -> str:
```

Plain English:
Start of the classifier. Docstring explains rule-based matching for now.

112-115

```
text = (question or "").lower().strip() / empty -> ESCALATION
```

Plain English:
Normalize input. Empty messages escalate instead of guessing.

117-133

```
escalation_clues list + return ESCALATION
```

Plain English:
Highest priority: human request, frustration, legal words.

135-149

```
returns_clues + RETURNS_REFUNDS_COMPLAINTS
```

Plain English:
Money and dissatisfaction lane - checked early on purpose.

support.py - Classifier and Escalation Packet

151-163

```
account_clues + ACCOUNT_HELP
```

Plain English:
Login / password / profile lane.

165-178

```
order_clues + ORDER_HELP (# see orders.py)
```

Plain English:
Classification only. Comment points to orders.py for lookup/status/tracking.

180-198

```
product_clues + PRODUCT_QUESTIONS
```

Plain English:
Product features and materials lane.

200-216

```
sleep_clues + SLEEP_GUIDANCE
```

Plain English:
General sleep tips lane (not medical care).

218-219

```
return ESCALATION # nothing matched
```

Plain English:
Safety net: do not invent a lane; hand off to a human.

222-224

```
def category_label(...)
```

Plain English:
Pretty-print helper with Escalation fallback for unknown codes.

227-229

```
def _contains_any(...)
```

Plain English:
Tiny private helper (leading underscore) used by escalation reason logic.

232-277

```
infer_escalation_reason(question)
```

Plain English:
Decide WHY we escalated: empty, legal, safety, requested human, stuck, refund, or no match.

280-286

```
priority_for_reason(reason)
```

Plain English:
Map reason to P1 critical, P2 high, or P3 normal.

289-322

```
build_escalation_packet(...)
```

Plain English:
Assemble the EscalationPacket with id, timestamp, notes, and transcript for operators.